
Introduction to Linux

Reference Document

6COSC019W

Dr Ayman El Hajjar

Academic Year: 2025/26



Please read this

- ➡ **NOTE-** In this lab, you only need to use the **Kali Linux machine** for most of the lab. Check "**Lab1 - Lab environment setup**" document on how to add the machine to VMware.
- ➡ In Section.8.1 of this lab, you will need the OWASP machine to test the network connectivity between the Virtual machines.
- ➡ This is **NOT** a step by step lab and it is **NOT** part of your assessment. It is provided as a **reference guide** to help you understand Linux, so that you can complete your assessed labs more confidently.

1 Introduction To Linux

- ➡ Linux is more than an OS. It's an idea where everybody grows together and there's something for everybody.
- ➡ For this reason, there are many flavours and distributions for Linux.
- ➡ The two most commonly used architecture for those distributions are either RPM based or Debian based.
 - ↳ **RPM** Red Hat Linux and SUSE Linux were the original major distributions that used the .rpm file format, which is today used in several package management systems.
 - ↳ **Debian:** Debian is a distribution that emphasises free software. It supports many hardware platforms. Debian and distributions based on it use the .deb package format and the dpkg package manager and its frontends (such as apt-get or synaptic)
 - ↳ Kali and Ubuntu, are Debian. In this lab we are using Kali Linux.

Important pointers

- ➡ In most cases, Linux error messages are self explanatory. Always read carefully what the terminal tells you.
- ➡ When you type a password in the terminal, nothing will appear on the screen. This is normal and is done to hide the length of the password.
- ➡ Linux is case sensitive. This means that upper case and lower case letters are treated as different characters.

1.1 File system tree

- ➡ Linux organises files using a hierarchical, tree-like directory structure.
- ➡ The root directory is at the top, with all other files and folders below it.
- ➡ All storage devices appear within this single directory tree.

```
(kali㉿kali)-[/]
$ tree -L 1-d /
/
├── bin → usr/bin
├── boot
├── dev
├── etc
├── home
├── initrd.img → boot/initrd.img-6.1.0
├── initrd.img.old → boot/initrd.img-6
├── .1.0-kali9-amd64
├── lib → usr/lib
├── lib32 → usr/lib32
├── lib64 → usr/lib64
├── libx32 → usr/libx32
├── lost+found
├── media
├── mnt
├── opt
├── proc
├── root
├── run
├── sbin → usr/sbin
├── srv
├── swapfile
├── sys
├── tmp
├── usr
├── var
├── vmlinuz → boot/vmlinuz-6.1.0-kali9
├── vmlinuz.old → boot/vmlinuz-6.1.0-k
└── al19-amd64

23 directories, 5 files
```

Figure 1.1: Kali Linux file system tree (tree command output)

- ➡ \ - Root (top level) directory
- ➡ \bin - Essential system commands
- ➡ \boot - Files used to start the system
- ➡ \dev - Hardware and device files
- ➡ \etc - System configuration files
- ➡ \home - User home directories
- ➡ \lib, \lib32, \lib64, \libx32 - System libraries and kernel modules
- ➡ \media - Mount points for removable devices
- ➡ \mnt - Temporary mount points
- ➡ \opt - Optional software packages
- ➡ \proc - Virtual system and process information
- ➡ \root - Home directory for the root user
- ➡ \run - Runtime data for processes and services
- ➡ \sbin - System administration commands
- ➡ \srv - Data for system services
- ➡ \sys - Hardware and kernel information
- ➡ \tmp - Temporary files
- ➡ \usr - User installed software and tools
- ➡ \var - Variable data such as logs and databases

2 Learning the shell

- ➞ The command line refers to the shell, which takes keyboard commands and passes them to the operating system.
- ➞ Most Linux systems use the bash shell.
- ➞ In a graphical environment, we use a terminal emulator to access the shell.
- ➞ This program is called **Terminal**.

2.1 Command Execution

- ➞ Commands run when you press Enter.
- ➞ Options change command behaviour.
- ➞ Short options use a single letter with a hyphen, e.g. **-a**, **-z**.
- ➞ Short options can be combined, e.g. **-az**.
- ➞ Some commands use long options with two hyphens, e.g. **--option**.
- ➞ Use **man <command>** to see all available options.

2.2 Simple Commands

- ➞ **date**, **df**, **free**: Show system information.
- ➞ Up arrow shows previous commands.
- ➞ **history** lists recent commands.

```
➤- history
```

- ➞ Use **exit** to close the terminal.
- ➞ Arguments follow the command, e.g. **cd /bin**.
- ➞ To execute multiple commands on a single line, separate them with semi-colons and press Enter only after the last one

```
➤- pwd ; date ; ls
```

3 Checking Your Login with whoami

- ➞ If you've forgotten whether you're logged in as root or another user, you can use the **whoami** command to see which user you're logged in as:

```
➤- whoami
```

3.1 Commands to navigate file systems

Command options

- ➡ Most commands use options consisting of a single character preceded by a dash, such as `-l`.
- ➡ Many commands, including those from the GNU Project, also support long options, consisting of a word preceded by two dashes.
- ➡ If you want to know what options are available for a specific command, you can read its manual page.
- ➡ **man**: For example, typing **man ls** will display the manual for the **ls** command, including all available options.

- ➡ The **pwd** command displays the current working directory.

```
➤ pwd
```

- ➡ When using a GUI, you can easily see which directory you are working in.
- ➡ When using the command line, you need to use a command to identify this.
- ➡ **ls**: Lists the files and directories in the current directory. It can also take a directory name as an argument.

```
➤ ls
```

- ➡ To list all files, including hidden files (those starting with a period):

```
➤ ls -a
```

- ➡ To display the results in long format, which shows additional useful information:

```
➤ ls -l
```

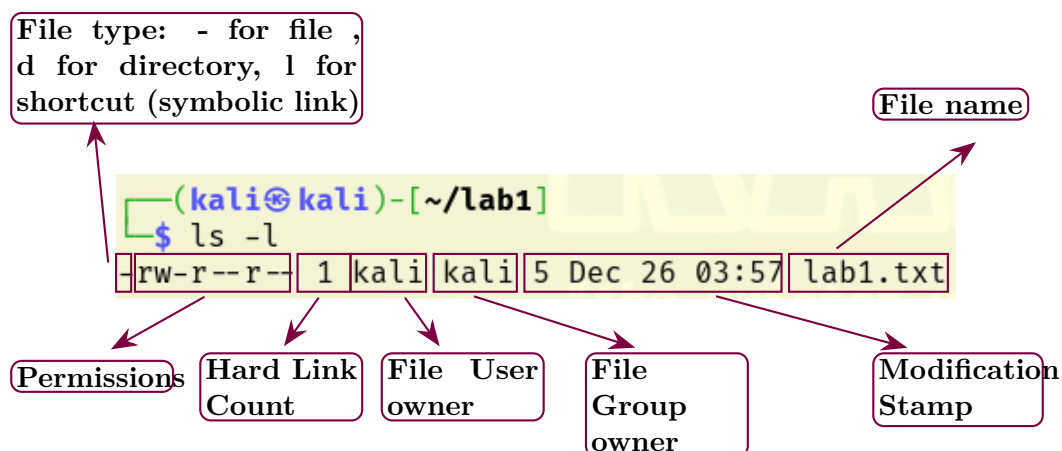


Figure 3.1: `ls -l` Output Command

4 Manipulating Files and Directories

➡ To change your working directory (where you are in the file system), use the **cd** command.

```
>- cd /bin
```

➡ This changes your working directory to the **bin** directory.

↳ To move to a specific directory, type its full path. For example, to navigate to the Pictures folder in your home directory:

```
>- cd /home/kali/Pictures
```

↳ or

```
>- cd ~/Pictures
```

➡ The **~** symbol represents the home directory of the current user.

➡ In this case, **~** represents **/home/kali**.

➡ Using **~** will always take you back to your home directory.

➡ There are also useful shortcuts for navigating with **cd**.

↳ To change to your home directory:

```
>- cd ~
```

↳ To move to the parent directory:

```
>- cd ..
```

↳ To return to the previous working directory:

```
>- cd -
```

↳ Before continuing, navigate to your Desktop folder so we can create a directory for the first lab.

```
>- cd ~/Desktop
```

➡ The **mkdir** command is used to create directories.

```
>- mkdir
```

↳ To create a directory called **lab1**:

```
>- mkdir lab1
```

➡ You can view the available options for **mkdir** using the manual:

```
>- man mkdir
```

-
- ➞ The **cp** command is used to copy files or directories.

```
>- cp
```

- ↳ To copy **item1** to **item2**:

```
>- cp item1 item2
```

- ↳ You can view the available options for **cp** using the manual:

```
>- man cp
```



Absolute vs. Relative paths

- ➞ An **absolute path** starts from the root directory, e.g. `/home/kali/Pictures`.
- ➞ A **relative path** depends on where you are, e.g. `Pictures`.

4.1 Text manipulation

- ➞ Viewing files: Use the **cat** command to display a text file in the terminal.

```
>- cat /etc/adduser.conf
```

- ↳ If the file is long, you may not see all of it. Use **head** or **tail** to view parts of the file.

- ➞ Use **head** to show the first few lines:

```
>- head /etc/adduser.conf
```

- ↳ Use **head -20** to show the first 20 lines:

```
>- head -20 /etc/adduser.conf
```

- ➞ Use **tail** to show the last lines of a file:

```
>- tail /etc/adduser.conf
```

- ↳ Use **tail -20** to show the last 20 lines:

```
>- tail -20 /etc/adduser.conf
```

- ➞ To display a file with numbered lines, use **nl**:

```
>- nl /etc/adduser.conf
```

- ➞ To display text directly in the terminal, use the **echo** command:

```
>- echo "this is my text"
```

- ➞ The **touch** command creates empty files or updates timestamps.

↳ To create an empty file:

```
>- touch lab1
```

↳ If the file already exists, **touch** updates its timestamp.

↳ To create multiple files at once:

```
>- touch lab1 lab2
```

↳ View all **touch** options using:

```
>- man touch
```

➡ Now move to the **lab1** folder:

```
>- cd ~/Desktop/lab1
```

➡ To append text to a file called **example1**:

```
>- echo 'This is my first line' » example1
```

↳ **echo 'This is my first line'** prints the text to the terminal.

↳ **» example1** appends the text to the file (or creates it if it does not exist).

➡ To check a file's type, use the **file** command:

```
>- file example1
```

➡ Use **less** to view large text files interactively:

```
>- less /etc/passwd
```

↳ **Space**: one page forward

↳ **b**: one page back

↳ **Enter / Down arrow**: down one line

↳ **Up arrow**: up one line

↳ **q**: quit

➡ The **mv** command is used to move or rename files.

↳ Rename a file:

```
>- mv example1 example2
```

↳ Move a file to another directory:

```
>- mv example2 ~/Desktop/example2
```

↳ Move and rename at the same time:

```
>- mv example2 ~/Desktop/lab1/example1
```

↳ View all **mv** options using:

```
>- man mv
```

➡ Move back to the Desktop:

```
>- cd ~/Desktop
```

➡ The **rm** command deletes files and directories.

↳ Delete a file:

```
>- rm example2
```

↳ Deleting a directory without options will fail:

```
>- rm lab1
```

↳ Use **-r** to delete a directory and its contents:

```
>- rm -r lab1
```

↳ View all **rm** options using:

```
>- man rm
```

4.2 Creating and editing text files

➡ To create or edit text files in Linux, you can use either a graphical text editor or a command-line editor.

➡ Mousepad is a simple graphical text editor. It can be used to create new files or edit existing ones.

↳ To create or edit a file with Mousepad:

```
>- mousepad filename.txt
```

➡ Nano is a command-line text editor that runs inside the terminal. It can also be used to create or edit files.

↳ To create or edit a file with Nano:

```
>- nano filename.txt
```

↳ In Nano, use **Ctrl + X** to exit and **Y** to save changes.

5 Permission

➡ Linux is a multiuser operating system, meaning multiple users can use the same computer at the same time.

-
- ↳ Users can also connect remotely using SSH and run commands.
 - ➡ To prevent users from interfering with each other, Linux uses file permissions and process isolation.
 - ➡ File and directory access is controlled using three permissions: read, write, and execute.
 - ➡ These permissions are shown in the output of the **ls** command.

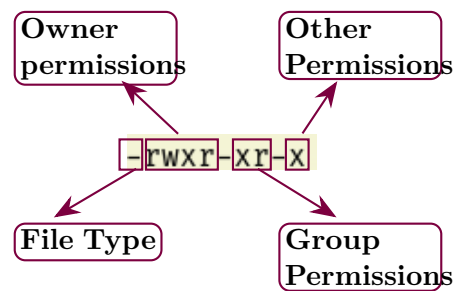


Figure 5.1: Permissions display

- ➡ There are three types of access permissions:
 - ↳ **r** (read) – Allows viewing directory contents (e.g. **ls**).
 - ↳ **w** (write) – Allows creating or deleting files in a directory.
 - ↳ **x** (execute) – Allows entering a directory (e.g. **cd**).
- ➡ There are also three types of users:
 - ↳ **Owner** – The user who owns the file.
 - ↳ **Group** – Users in the same group as the owner.
 - ↳ **Other** – All other users on the system.
- ➡ Files and directories can have different permission combinations for each user type.
 - ↳ Directory permissions control actions such as entering, listing, or modifying files inside the directory.
- ➡ To change file or directory permissions, use the **chmod** command.
 - ↳ Only the file owner or the superuser can change permissions.
 - ↳ **chmod** supports two methods: octal numbers and symbolic notation.

Your table is already fine and does not need changing.

Now the **chmod** example, summarised and clearer:

- ➡ To set the following permissions on a file called **example1**:
 - ↳ **Owner**: read, write, execute

↳ **Group:** read, write

↳ **Other:** read

➡ Use the following command:

```
❯ chmod 764 example1
```

↳ 7 = read + write + execute (4+2+1)

↳ 6 = read + write (4+2)

↳ 4 = read

5.1 Super user command

➡ Some tasks require superuser (administrator) privileges.

↳ These include installing software, editing system files, and accessing protected directories.

➡ On Linux, these tasks are performed using the **sudo** command.

↳ **sudo** temporarily gives you higher privileges to run a command safely.

➡ Let's try to copy **example1** into a protected directory:

```
❯ cp example1 /usr/bin
```

↳ You will see a **Permission denied** error.

➡ Now run the same command using **sudo**:

```
❯ sudo cp example1 /usr/bin
```

↳ You will be asked for your password.

↳ When typing a password in the terminal, nothing appears on the screen.

↳ This is normal and hides the length of your password.

6 Pipes and connecting things

Before continuing, you need to download the module repository from GitHub.

↳ Use the **git clone** command to pull the repository from GitHub.

```
❯ git clone https://github.com/azelhajjar/6COSC019W.git
```

↳ This downloads the **6COSC019W** folder, which contains the files needed for the next lab tasks.

-
- ➡ Pipes (|) send the output of one command to another command.

```
>- echo "this is my first Linux lab" | wc
```

- ↳ **echo** prints the text "this is my first Linux lab".

- ↳ The pipe | sends this output to **wc**.

- ↳ **wc** counts lines, words, and characters.

- ➡ The output shows: 1 line, 6 words, and 27 characters.

- ➡ In Cyber Security, you often work with large text files and logs.

- ↳ It can be difficult to find specific information in large outputs.

- ➡ The **grep** command searches for text patterns in files.

- ↳ For example, to find the user **kali** in **/etc/passwd**:

```
>- grep "kali" /etc/passwd
```

- ↳ If there are many results, you can add line numbers using **-n**:

```
>- grep -n "kali" /etc/passwd
```

- ↳ The **-n** option shows the line number for each match.

- ➡ The **awk** command is used to extract specific data from files using rules.

- ➡ Make sure you have downloaded the **6c0sc019w** repository.

- ↳ Navigate to the folder:

```
>- cd 6C0SC019W
```

- ➡ The log file contains usernames and password strength data stored as SHA256 hashes.

- ➡ To display the second column in **users.log**:

```
>- awk -F '\t' '{print $2}' users.log
```

- ↳ **-F '\t'** sets the column separator to a tab.

- ↳ **print \$2** displays the second column.

- ↳ **users.log** is the file being analysed.

- ➡ To display all data for the user **John**:

```
>- awk '$1 == "John" {print $0}' users.log
```

- ↳ **\$1 == "John"** checks the first column for the name John.

- ↳ **print \$0** prints the entire matching line.

➡ Note: **awk** is a powerful command that allows you to create flexible text-processing rules.

7 Adding and removing software

➡ When doing your labs, sometimes you will have to add or install some software.

➡ In Linux, the default software manager is called Advanced Packaging Tool or **apt**.

↳ You can use either **apt** or **apt-get**, but **apt-get** has more functionality.

➡ It is useful to always update the software repository list as it is regularly updated with new software databases.

```
➤ sudo apt-get update
```

↳ We update the repository to ensure that you are using the latest repository database.

➡ If we want to install an application, as long as it is available in the apt repository, we can install it using **apt-get install**.

```
➤ sudo apt-get install chromium
```

↳ This installs the Chromium browser, an alternative to Firefox.

```
➤ sudo apt-get install git
```

↳ This installs Git, which allows you to download software from GitHub.

↳ You can replace **git** with the name of the software you want to install.

➡ You can remove software in a similar way using **remove** instead of **install**.

```
➤ sudo apt-get remove chromium
```

↳ This removes Chromium.

↳ Some software leaves configuration files behind. The **purge** option removes these completely.

```
➤ sudo apt-get purge chromium
```

8 Networking

➡ We need to know the IP address of our machine. We can either use **ifconfig** or **ip address**.

```
➤ ifconfig
```

```
➤ ip address
```

↳ or **ip add** or **ip a**

➡ In this figure, we used the **ifconfig** command to list all the interfaces that are active on our Kali machine.

```
(kali@kali)-[~]
$ ifconfig
br-a80e6e498b93: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 172.18.0.1 netmask 255.255.0.0 broadcast 172.18.255.255
    inet6 fe80::42:2bff:fe5a:42bc prefixlen 64 scopeid 0<link>
    ether 02:42:2b:5a:42:bc txqueuelen 0 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

br-e467d21605c1: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 172.19.0.1 netmask 255.255.0.0 broadcast 172.19.255.255
    inet6 fe80::42:1aff:fe72:a5f0 prefixlen 64 scopeid 0<link>
    ether 02:42:1a:72:a5:f0 txqueuelen 0 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

docker0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 172.17.0.1 netmask 255.255.0.0 broadcast 172.17.255.255
    inet6 fe80::42:ccff:fe4a:9925 prefixlen 64 scopeid 0<link>
    ether 02:42:cc:4a:99:25 txqueuelen 0 (Ethernet)
    RX packets 49 bytes 3109 (3.0 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 39 bytes 4818 (4.7 KiB)
    TX errors 0 dropped 2 overruns 0 carrier 0 collisions 0

eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.144.128 netmask 255.255.255.0 broadcast 192.168.144.255
    inet6 fe80::27f1:3d4b:88b0:6980 prefixlen 64 scopeid 0<link>
    ether 00:0c:29:d9:07:ba txqueuelen 1000 (Ethernet)
    RX packets 51 bytes 7461 (7.3 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 25 bytes 3482 (3.4 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 8 bytes 480 (480.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 8 bytes 480 (480.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

veth09880da: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet6 fe80::744d:21ff:fe6f:1b46 prefixlen 64 scopeid 0<link>
    ether 76:4d:21:6f:1b:46 txqueuelen 0 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 26 bytes 3873 (3.7 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

veth233f2e4: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet6 fe80::2458:29ff:fe27:290c prefixlen 64 scopeid 0<link>
    ether 26:58:29:22:39:0c txqueuelen 0 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 26 bytes 3941 (3.8 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

veth3f42a66: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet6 fe80::bc84:1eff:fe21:f419 prefixlen 64 scopeid 0<link>
    ether be:84:1e:21:f4:19 txqueuelen 0 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 26 bytes 3941 (3.8 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

➡ Both commands will list all your network interfaces and the different IP addresses allocated for each interface.

➡ We can see many interfaces. This is because we use Docker in future labs. Docker creates its own **docker service interface** to control containers, and each docker instance can create a **docker instance virtual interface**.

➡ We also have the **localhost interface IP 127.0.0.1** for services running locally. It is

often used to test services locally before deploying them.

➡ For each interface listed, you get information such as the IP address, subnet, and broadcast address. You also get packet statistics (for example transmitted and dropped packets) and information about the interface type.

➡ In this case we have two types: Ethernet and the localhost loopback. If a wireless card was connected, you would also see a wireless interface.

💡 As you can see, **ifconfig** shows information about all interfaces, but in most labs we only need the IP address of the **eth0** interface (the Kali virtual machine interface).

➡ To do this, we can use **ifconfig** for a specific interface, such as **eth0**:

```
>- ifconfig eth0
```

➡ or

```
>- ip a show eth0
```

The screenshot shows two terminal windows. The top window is titled 'Kali VMware Ethernet adapter' and shows the output of 'ifconfig eth0' for a 'Host-Only' network. The IP address '192.168.144.128' is highlighted with a yellow box and labeled 'Host-Only IP address'. The MAC address '00:0c:29:d9:07:ba' is also highlighted with a yellow box. The bottom window shows the output of 'ifconfig eth0' for a 'NAT' network. The IP address '192.168.160.128' is highlighted with a yellow box and labeled 'NAT IP address'. The MAC address '00:0c:29:d9:07:ba' is also highlighted with a yellow box and labeled 'eth0 MAC address'. A yellow line connects the MAC address in the NAT window to the MAC address in the Host-Only window.

```
(kali@kali)-[~]
$ ifconfig eth0
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.144.128 netmask 255.255.255.0 broadcast 192.168.144.255
    inet6 fe80::27f1:3d4b:85b0:6980 prefixlen 64 scopeid 0x20<link>
    ether 00:0c:29:d9:07:ba txqueuelen 1000 (10.0 KiB)
    RX packets 60 bytes 9740 (9.5 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 29 bytes 3696 (3.6 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

(kali@kali)-[~]
$ ifconfig eth0
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.160.128 netmask 255.255.255.0 broadcast 192.168.160.255
    inet6 fe80::27f1:3d4b:85b0:6980 prefixlen 64 scopeid 0x20<link>
    ether 00:0c:29:d9:07:ba txqueuelen 1000 (10.0 KiB)
    RX packets 30 bytes 14395 (14.0 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 11 bytes 1144 (11.4 KiB)
    TX errors 0 dropped 3 overruns 0 carrier 0 collisions 0
```

➡ In our lab environment, most of the time we need to be connected to our Host Only network. This is our private isolated network used to conduct penetration tests in a safe environment.

➡ It is important to note that when we change the network settings while the virtual machine is running, we must tell the DHCP server to dynamically allocate a new IP address (Fig.8.1).

➡ Bringing the interface down and up resets the connection, but does not always request a new IP address from DHCP. it depends on the state of the eth0 link interface.

```
➤ sudo ifconfig eth0 down
```

➡ This will bring down the eth0 interface (disable it).

```
➤ sudo ifconfig eth0 up
```

➡ This will bring up the eth0 interface (enable it).

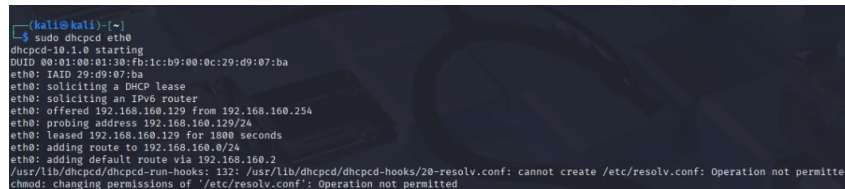
➡ To force a new IP allocation, the DHCP client or NetworkManager must be restarted.

↳ To force a new IP allocation using the DHCP service:

```
➤ sudo dhcpcd eth0
```

↳ You can also force a new IP address to be assigned by restarting the NetworkManager service:

```
➤ sudo systemctl restart NetworkManager
```



```
(kali@kali)~$ sudo dhcpcd eth0
dhcpcd-10.1.0 starting
UUID 00:11:00:01:20:03:1c:b9:00:0c:29:d9:07:ba
eth0: IAID 29:d9:07:ba
eth0: soliciting a DHCP lease
eth0: soliciting an IPv6 router
eth0: offered 192.168.160.129 from 192.168.160.254
eth0: probing address 192.168.160.129/24
eth0: leased 192.168.160.129 for 1800 seconds
eth0: adding route to 192.168.160.0/24
eth0: adding default route via 192.168.160.2
/usr/lib/dhcpcd/dhcpcd-run-hooks: 132: /usr/lib/dhcpcd/dhcpcd-hooks/20-resolv.conf: cannot create /etc/resolv.conf: Operation not permitted
chmod: changing permissions of '/etc/resolv.conf': Operation not permitted
```

Figure 8.1: dhcpcd eth0

➡ To check connectivity, we can use the **ping** command with the IP address of the host machine.

```
➤ ping 192.168.144.1
```

➡ This sends ICMP request messages to your host machine IP **192.168.56.1**. If it is reachable, it will respond with ICMP replies.

➡ To stop ping, press **ctrl** and **c**, otherwise it will run continuously.

```
➤ ping -c 5 192.168.144.1
```

➡ You can specify the number of requests using the **-c** option. In this example, it will stop after sending 5 requests.

8.1 Test connectivity between Virtual machines

OWASP VM

- ➡ Before you start the OWASP VM, make sure it is attached to **Host-Only** network.
- ➡ Start the machine and note its IP address.


```
root@owaspbwa:~# ifconfig eth0
eth0      Link encap:Ethernet  HWaddr 00:0c:29:8e:fb:6e
          inet addr:192.168.144.129  Bcast:192.168.144.255  Mask:255.255.255.0
          inet6 addr: fe80::20c:29ff:fe8e:fb6e/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:73 errors:0 dropped:0 overruns:0 frame:0
          TX packets:91 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:11942 (11.9 KB)
          Interrupt:18 Base address:0
```

OWASP IP address

➡ To check connectivity between devices:

↳ From Kali- ping the OWASP VM:

```
>- ping 192.168.144.129
```

↳ From OWASP- ping the Kali VM::

```
>- ping 192.168.144.129
```

➡ when we selected Host-Only network we created a private network between our VMs that is completely isolated from the external world.

➡ Your host machine also have a virtual Box interface that connects your device to the private network.

↳ To ping your host machine (physical) machine from either OWASP or Kali VMs:

```
>- ping 192.168.144.1
```